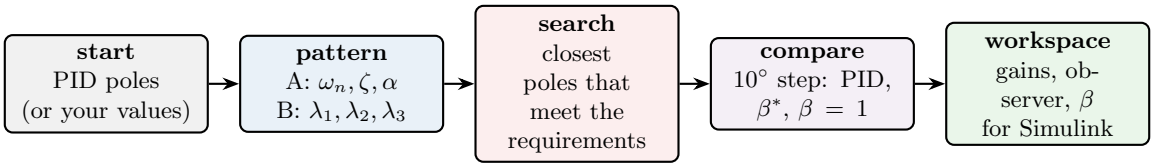
Pole placement for the CNC plotter

theory behind the `pp_design` folder: model, polynomial matching, β, requirements, search, approaches A and B

1 The workflow at a glance



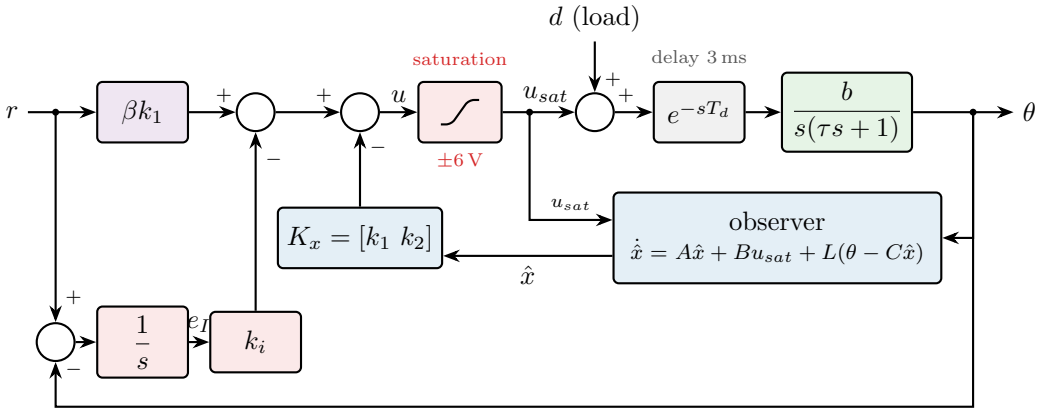
The same steps hold for both approaches. Only the *pattern* (how the three poles are written) differs: Sections 2–6 are common, Section 7 compares A and B. `explore_poles.m` shows the same quantities live: drag a pole (a complex one drags its conjugate), press `c` for a pair + real pole or `r` for three real poles, or type the poles.

2 Plant and control scheme

Motor (one per axis, identified with the link attached), angle in encoder degrees:

$$\tau\ddot{\theta} + \dot{\theta} = b\,(u + d), \qquad G(s) = \frac{b}{s(\tau s + 1)}, \qquad b = -\tfrac{180}{\pi}k, \qquad x = \begin{bmatrix}\theta \\ \omega\end{bmatrix},\ A = \begin{bmatrix}0 & 1 \\ 0 & -\frac{1}{\tau}\end{bmatrix},\ B = \begin{bmatrix}0 \\ \frac{b}{\tau}\end{bmatrix},\ C = [1 \quad 0].$$

Only θ is measured, so an **observer** estimates $\hat{x} = [\hat{\theta}, \hat{\omega}]$. An **integrator** of the error e_I removes steady errors, and the reference enters with a **weight** β.



Control law (the same in the scripts and in Simulink)

$$u = \underbrace{\beta k_1 r}_{\text{reference weight}} - \underbrace{k_1\hat{\theta} - k_2\hat{\omega}}_{\text{state feedback}} - \underbrace{k_i e_I}_{\text{integral}}, \qquad \dot{e}_I = r - \theta.$$

The saturation and the delay are part of the real loop; the next box explains them.

The two extra blocks: saturation and delay

Saturation (± 6 V). The amplifier cannot give more than 6 V. It is already in the Simulink models (block `Saturation1` in each `pp + luenberg` subsystem). It is *not linear*, so the pole-placement formulas cannot include it. The design avoids it instead: the requirement *peak voltage* $\leq 6\ V$ keeps the 10° step inside the linear range. If the voltage still saturates (a bigger step), the integrator would keep growing (*windup*); the anti-windup gain K_{aw} feeds back $u_{sat} - u$ into the integrator and stops it. The observer receives u_{sat}, the voltage really applied.
Delay ($T_d = 1.5\,T_s = 3$ ms). The controller is digital and runs every $T_s = 2$ ms: it reads the encoder, computes u, and the voltage is then held constant until the next sample. The hold delays the signal on average by $T_s/2$, and the new value is applied one sample later (T_s): together about $1.5\,T_s$. A delay does not change the gain, it only removes phase: ωT_d radians at frequency ω (at the crossover ≈ 35 rad/s: 6°). This is why the phase margin is computed with it. It is not a block of the models: it comes from the sampling of the real hardware. To see it in `Model_pp_obs.slx`, add a *Transport Delay* of 0.003 s in front of the motor model.

How the scripts simulate the delay and the load (`utils/closed_loop.m`)

- **Delay**: the motor model gets `P.InputDelay = 0.003`: the voltage reaches the motor 3 ms later. MATLAB treats this delay exactly in `step` (time response) and in `margin` (phase margin).
- **Load (disturbance)**: a summing junction `sumblk('ut = u + d')` adds a voltage d to the controller voltage before the motor. A step of $d = 0.5$ V gives the curve `y_d`. It is only plotted by `explore_poles`: it is *not* one of the six requirements, and β does not change it.
- **Saturation**: not simulated (the model stays linear); the requirement *peak voltage* $\leq 6\ V$ checks that the step never reaches it. In Simulink the saturation block is real.

3 Pole placement by polynomial matching

Result

With $\hat{x} = x$ the closed loop has the characteristic polynomial

$$\Delta(s) = s^3 + a_2 s^2 + a_1 s + a_0, \qquad a_2 = \frac{1 + bk_2}{\tau}, \quad a_1 = \frac{bk_1}{\tau}, \quad a_0 = -\frac{bk_i}{\tau}.$$

Choosing the poles p_1, p_2, p_3 fixes $\Delta(s) = (s - p_1)(s - p_2)(s - p_3)$, hence the gains

$$\boxed{k_1 = \frac{\tau a_1}{b}, \qquad k_2 = \frac{\tau a_2 - 1}{b}, \qquad k_i = -\frac{\tau a_0}{b}} \qquad a_2 = -\sum \mathrm{Re}\, p_i, \quad a_1 = \textstyle\sum_{i<j} p_i p_j, \quad a_0 = -p_1 p_2 p_3.$$

Short proof

States $[\theta, \omega, e_I]$, $u = -k_1\theta - k_2\omega - k_i e_I$, $\dot{e}_I = -\theta$ (with $r = 0$): $M = \begin{bmatrix} 0 & 1 & 0 \\ -\frac{bk_1}{\tau} & -\frac{1+bk_2}{\tau} & -\frac{bk_i}{\tau} \\ -1 & 0 & 0 \end{bmatrix}$. Expanding $\det(sI - M)$ along the first row gives $s^3 + \frac{1+bk_2}{\tau} s^2 + \frac{bk_1}{\tau} s - \frac{bk_i}{\tau}$. Each gain appears in one coefficient only and $b \neq 0$: the gains exist and are unique. The coefficients of $(s - p_1)(s - p_2)(s - p_3)$ follow by expanding the product (Vieta); they are real because complex poles come in conjugate pairs.

In the code: `acker` does the same matching

`K = acker(Ai, Bi, p)` returns the gain of $u = -Kz$, $z = [\theta, \omega, e_I]$, that puts the eigenvalues of $A_i - B_i K$ in p (Ackermann's formula, MATLAB Control Toolbox). Since $K = [k_1,\ k_2,\ k_i]$, it gives *the same numbers* as the formulas above: for the poles $-31.2 \pm 10.8j$, -7 both give $k_1 = -0.6941$, $k_2 = -0.0194$, $k_i = 3.4689$ (motor 3165). The formulas are kept here to understand *why* each gain moves each coefficient; the code uses `acker` because it is shorter to read. The observer uses the same function on the transposed system: `L = acker(A', C', p_obs)'`, because $\mathrm{eig}(A - LC) = \mathrm{eig}(A^T - C^T L^T)$.

Observer gain (same idea)

$\det(sI - A + LC) = s^2 + (l_1 + \frac{1}{\tau})s + (\frac{l_1}{\tau} + l_2)$. For observer poles $-o_1, -o_2$: $l_1 = o_1 + o_2 - \frac{1}{\tau}$, $l_2 = o_1 o_2 - \frac{l_1}{\tau}$. Choice: $o_1 = \min(6 \max|p_i|, 250)$, $o_2 = 1.25\, o_1$ (250 rad/s $= 0.5/T_s$).

Separation and PID start

The estimation error obeys $\dot{\varepsilon} = (A - LC)\varepsilon + Bd$: r cancels, so the reference sees only the three controller poles. The PID is the state feedback $K = [K_p, K_d, -K_i]$: its poles are a natural starting point.

4 The reference weight β: a zero in the reference path

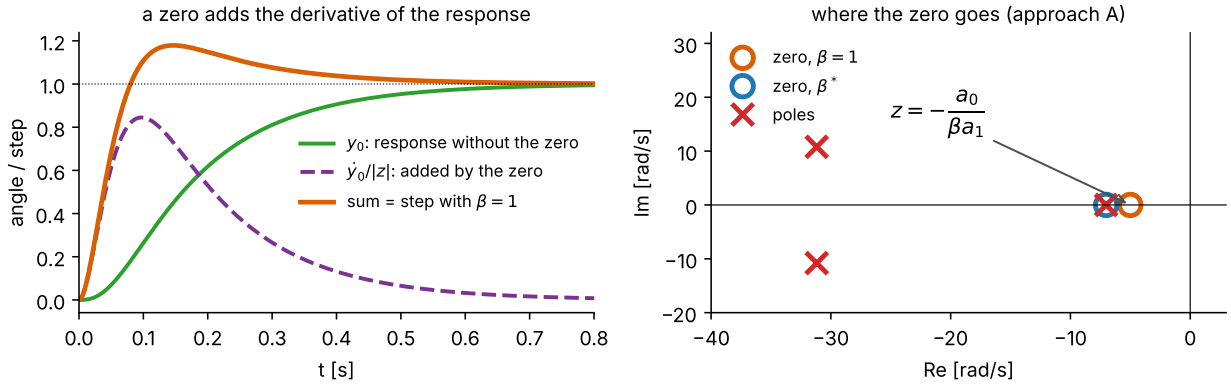
Reference to angle

$$\frac{\Theta(s)}{R(s)} = \frac{\beta a_1 s + a_0}{\Delta(s)}, \qquad \text{one zero at } z = -\frac{a_0}{\beta a_1}, \qquad \frac{\Theta(0)}{R(0)} = 1 \text{ for every } \beta.$$

Short proof

With $d = 0$ the observer is exact, $\hat{x} = x$. Laplace transform of $\tau\ddot{\theta} + \dot{\theta} = bu$ with $U = \beta k_1 R - k_1\Theta - k_2 s\Theta - k_i(R - \Theta)/s$, times s: $[\tau s^3 + (1 + bk_2)s^2 + bk_1 s - bk_i]\Theta = b(\beta k_1 s - k_i)R$. Divide by τ and use Section 3.

What the zero does. Write $\frac{\beta a_1 s + a_0}{\Delta} = \frac{a_0}{\Delta}\left(1 + \frac{s}{|z|}\right)$. The step response is $y = y_0 + \dot{y}_0/|z|$: the response *without* zero plus its derivative, scaled by $1/|z|$. The derivative is a positive bump at the start, so the zero makes the response faster and, if it is slow (small $|z|$), pushes it above the target.



β	zero	step	voltage at $t = 0^+$
$\beta = 1$ **(no weight)**	$\lvert z\rvert = a_0/a_1$, slower than every pole	overshoot, also with real poles	$k_1 R$ (largest)
$\beta^* = \dfrac{a_0}{a_1 \lambda_{slow}}$	on the slowest real pole: cancels it	follows the other two poles	$\beta^* k_1 R$
$\beta = 0$ **(not used)**	none	slowest pole only: very slow	0

With real poles $|z|_{\beta=1} = 1/\sum_i \lambda_i^{-1} < \min_i \lambda_i$ (divide a_0/a_1 by $\prod \lambda_i$): the zero is always the slowest element, which is why $\beta = 1$ overshoots even with three real poles.

Why $\beta = 0$ is not an option

With $\beta = 0$ the reference enters *only through the integrator*: $u = -k_1\hat{\theta} - k_2\hat{\omega} - k_i e_I$. Then $\Theta/R = a_0/\Delta(s)$ has no zero, and the angle reaches r only as fast as the integral action, i.e. as the **slowest pole**. With the slowest pole at $\lambda_{min} = 2$ rad/s the step settles in about $4/2 = 2$ s, ten times slower than the PID, and on a path the lag $\frac{a_1}{a_0}v$ is the largest possible (about 10° at 50°/s in our model). It avoids overshoot only by being slow, so it is **not useful for our control** and it has been removed from all the scripts. The scripts compare **PID,** β^* **and** $\beta = 1$: β^* for steps, $\beta = 1$ for paths with feedforward.

Should we worry about zeros?

- **Left-half-plane zeros** (ours: $a_0, a_1 > 0$ and $\beta \geq 0$) are harmless for stability. They only shape the step.
- **Right-half-plane zeros** are the dangerous ones (the response first goes the wrong way, bandwidth is limited). They would appear only with $\beta < 0$: never use it.
- β is *outside* the loop: poles, margins, chatter and the response to a load are the same for every β.
- Cancelling a pole with β^* hides it from the step only. The pole is still there for loads and model errors, and the cancellation is exact only for the nominal k, τ. So the hidden pole must not be too slow: requirement *slowest pole* $\geq \lambda_{min}$ (`req.lam_min`).
- On a path (ramp of speed v) a weight $\beta < 1$ gives a lag $(1-\beta)\frac{a_1}{a_0}v$: for trajectories use $\beta = 1$.

5 The requirements kept

Six numbers, all in `utils/pp_setup.m`. The design is checked on **both motors**; the worst one counts.

requirement	limit	why
overshoot on a 10° step	$\leq 2\%$	a path corner must not be overshot
settling time (2% band)	$\leq 0.2\,\mathrm{s}$	the response must be faster than needed by the path
peak voltage on the step	$\leq 6\,\mathrm{V}$	no saturation: the linear design stays valid
phase margin (with the 3 ms delay)	$\geq 45°$	robustness to model errors and to the delay
chatter voltage (one encoder count)	off (`Inf`)	fast poles amplify the encoder noise: heating, buzzing (use e.g. 0.2 V)
slowest pole	$\geq \lambda_{min} = 2\,\mathrm{rad/s}$	speed of the integral action: loads and model errors (also the pole hidden by β^*) are removed in time

Not kept: gain margin (always large here), load-step error (no disturbance in the design), path tests (done on the robot).

6 The search: cost function

The search moves the poles *as little as possible* from the start until every requirement is met.

Cost minimised by `fminsearch`

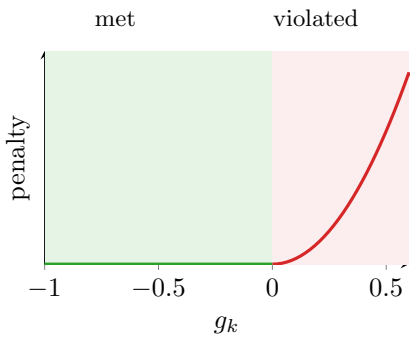
$$J(x) = \underbrace{\sum_{i=1}^{3} \left(\log|p_{(i)}(x)| - \log|p^0_{(i)}| \right)^2}_{\text{stay close to the start}} + \underbrace{w \sum_{k=1}^{6} \max\left(0,\, g_k(x)\right)^2}_{\text{penalty: only violated requirements}}, \qquad w = 10^4.$$

x = the pattern parameters (A: $\log\omega_n, \log\zeta, \log\alpha$; B: $\log\lambda_1, \log\lambda_2, \log\lambda_3$), p^0 = start poles (PID or your values), (i) = sorted by speed.

Normalised violations. For an upper limit $g_k = \dfrac{\text{value}}{\text{limit}} - 1$, for a lower limit $g_k = 1 - \dfrac{\text{value}}{\text{limit}}$. So $g_k \leq 0$ means met and $g_k = 0.1$ means 10% beyond the limit: all requirements are compared on the same scale.

Why logarithms. $\log|p| - \log|p^0| = \log(|p|/|p^0|)$: moving a pole from 10 to 20 rad/s costs as much as from 20 to 40. Only ratios matter, and $|p| = e^x$ is always positive.

Why sorting. For sorted sequences $\sum(a_i - b_i)^2$ is the smallest over all pairings (rearrangement inequality): each new pole is compared with the closest old one.



Properties of the penalty

$\max(0,g)^2 = 0$ exactly when $g \le 0$, so on the designs that meet everything J is only the distance: among them the search returns the closest to the start. The penalty is smooth at $g = 0$ (derivative $2\max(0,g)$). With a large w the remaining violation at the optimum is about $\mu/(2w)$ (μ = Lagrange multiplier), i.e. negligible. The requirements (overshoot, settling) are not smooth in the poles, so the derivative-free Nelder–Mead method is used and restarted until every $g_k \le 10^{-3}$.

How the search decides which pole to move

Nelder–Mead (`fminsearch`) keeps a small set of trial points ($n+1 = 4$ for 3 parameters). At every step it replaces the worst point by its reflection through the others (or a longer or shorter step), so the set rolls downhill on J. It does not know which pole "should" move: a parameter changes only if this lowers J. Since $w = 10^4$, fixing a violation is always worth more than staying close, so the poles move *just enough* to meet every requirement, and the cheapest move (in log distance) wins.

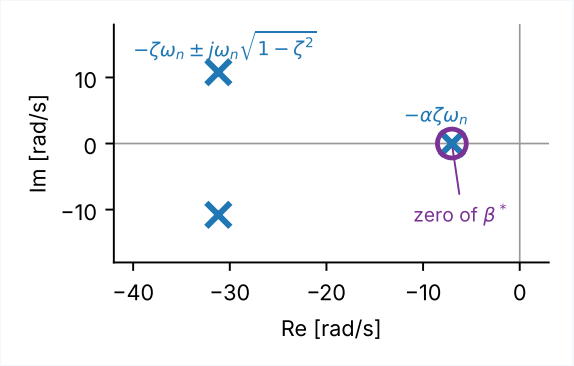
Same algorithm, different coordinates.

- **A**: $x = [\omega_n, \zeta, \alpha]$. ω_n scales the whole pattern (pair *and* real pole, since the real pole is $\alpha\zeta\omega_n$), ζ turns the pair (damping), α moves only the real pole. The pair always moves as a pair.
- **B**: $x = [\lambda_1, \lambda_2, \lambda_3]$: each real pole moves on its own.

Why the slow pole moves so much. No pole belongs only to the integrator: all three are poles of θ, ω and e_I. The slowest real pole is the one that sets how fast the integral action removes an error. The PID has one slow pole (a few rad/s, ≈ 2.6 rad/s in our analysis). With β^* it is hidden in the 10° step, so overshoot and settling would not move it, but the requirement *slowest pole* $\ge \lambda_{min}$ does: a pole below λ_{min} costs $10^4(1 - |p|/\lambda_{min})^2$, while moving it costs only $\log^2(\lambda_{min}/|p|)$. So the search moves it up to λ_{min} (2 rad/s by default), and further only if another requirement asks for it.

7 The two approaches

A: complex pair + real pole



Parameters ω_n (speed), ζ (damping), α (real pole / real part of the pair).

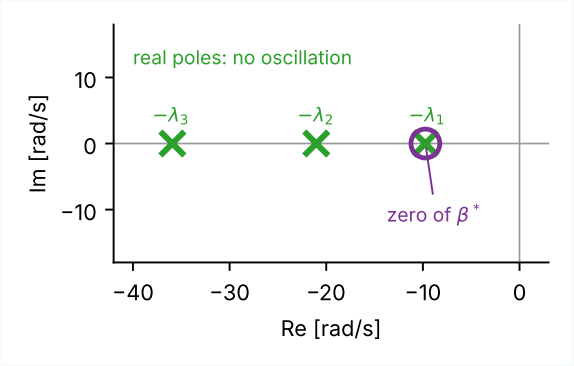
Weight $\beta^* = \dfrac{1}{1+2\alpha\zeta^2}$ puts the zero (circle) on the real pole:

$$\frac{\Theta}{R} = \frac{\omega_n^2}{s^2 + 2\zeta\omega_n s + \omega_n^2}$$

the classic second-order step (overshoot set by ζ).

Idea keep the familiar ω_n, ζ design, hide the slow integral pole with β^*.

B: three real poles



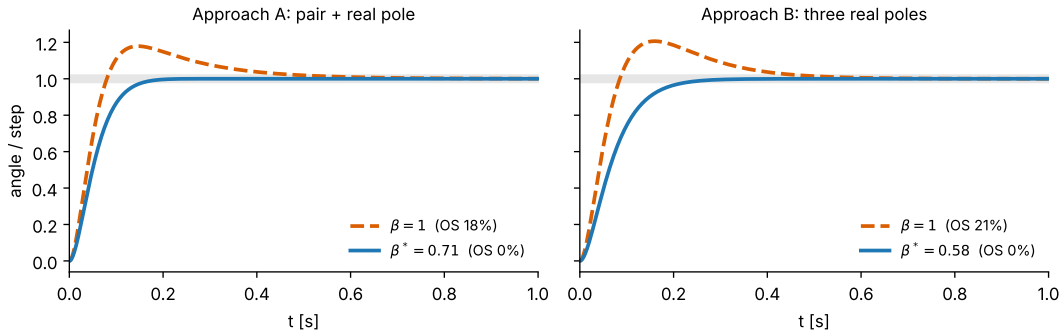
Parameters the three speeds $\lambda_1 < \lambda_2 < \lambda_3$.

Weight $\beta^* = \dfrac{a_0}{a_1\lambda_1}$ puts the zero on the slowest pole:

$$\frac{\Theta}{R} = \frac{\lambda_2\lambda_3}{(s+\lambda_2)(s+\lambda_3)}$$

no oscillation, and the slow λ_1 is hidden from the step. For paths: $\beta = 1$.

Idea no oscillating mode, each pole moves on its own; without a complex pair the rise is a bit slower.



Example poles (earlier design): A $\omega_n = 33$, $\zeta = 0.945$, $\alpha = 0.225$; B $-9.7, -21.0, -35.9$ rad/s. Model without delay.

	A: pair + real	B: three real
search variables	ω_n, ζ, α	λ_1, λ_2, λ_3
β for steps	β^* (cancels the real pole)	β^* (cancels λ_1)
β for paths	1	1
overshoot (with β^*)	set by ζ (2% at $\zeta \approx 0.78$)	none (two real poles)
voltage kick at the step	$\beta^* k_1 R$	$\beta^* k_1 R$
weak point	relies on an exact cancellation	exact cancellation; slower rise
same for both	model, polynomial matching, observer, requirements, cost function, Simulink variables	

How they differ, in one sentence: A shapes the step with a complex pair, B avoids any oscillation with three real poles; both use β^* to hide the slow pole on steps and $\beta = 1$ on paths. The loop structure is the same, so robustness (phase margin, chatter) is judged in the same way.

8 Using more of the plant: voltage and speed

Where the voltage goes

- **Step**: at $t = 0^+$ the voltage is $u = \beta k_1 R$. It grows with the step size R and with $k_1 = \tau a_1 / b$; a_1 grows like the square of the pole speeds, so *poles twice as fast need four times the voltage.*
- **Path**: the voltage is mostly the feedforward $(\tau \ddot{r} + \dot{r})/b$: at 50°/s it is about 0.6 V.
- **Top speed**: in steady state $\dot{\theta} = k\,u$; with $u = 6$ V the joint turns at $k \cdot 6 \approx 8.6$ rad/s ≈ 490°/s, about ten times the path speed used now.

What limits it

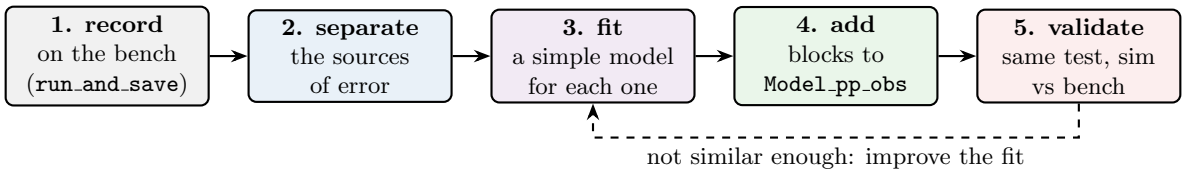
- **Headroom**: feedforward + feedback must stay below 6 V; keep 1–2 V free for corrections.
- **Current**: $R_m = 2.6\,\Omega$: at stall 6 V is 2.3 A, above the 1 A continuous limit (fine for short peaks only).
- **Chatter**: faster poles mean larger k_2 and a faster observer: more noise voltage.
- **Delay**: 3 ms removes ωT_d of phase; for 45° of margin the crossover must stay below roughly 80 rad/s.
- **Model**: identified with small signals; backlash and the linkage matter more at high speed.

How to push it

1. **Faster paths first**: shorter trajectory times use more feedforward voltage; the controller does not change. Check in the logs (`validate_controllers.m`) that the peak stays below about 5 V.
2. **Faster poles**: lower `req.ts_max` (e.g. 0.25 s) in `pp_setup.m`; the search then uses more voltage. If chatter or phase margin become binding, that is the real limit of the hardware.
3. **Bigger steps**: set `req.R` to the step really used, so the 6 V check is honest.
4. The 6 V limit is `req.Vmax` in `pp_setup.m` (and `Vmax` in `parameters.m` for Simulink): check the VoltPAQ-X2 and SRV02 manuals before raising it.

9 Proposal: model the noise of the real plant and simulate it

Goal. Today the model is ideal: an exact angle, no friction, no noise. The real motor has an encoder that counts in steps of 0.088°, friction in the gearbox, a link that pushes back, and electrical noise. If we measure these effects once on the bench and add them to `Model_pp_obs.slx`, the simulation shows chatter and tracking errors close to the real ones, before we touch the robot.



1. Record (CNC_pp_obs.slx, data in data/lab)

- **Hold still** 10 s with the controller on and r constant: log θ and u.
- **Motor off** ($u = 0$): log θ. Only the encoder is left.
- **Slow voltage ramp** from 0 V: the voltage at which the motor starts to move is the friction.
- **Constant speed** at 3–4 voltages: speed ripple, backlash.
- **Multisine** of the identification (`model_id`): data for the residual.

2. Separate the sources

- **Encoder**: θ is always a multiple of $q = 360/4096$: this is not random, it is a rounding.
- **Residual**: simulate $G(s)$ with the logged u and compute $e = \theta_{meas} - \theta_{model}$: what the model misses (friction, backlash, the link).
- **Spectrum**: `pwelch(e)` and `pwelch(u)` at rest: where the noise lives in frequency; `histogram(e)`: its size.

3. Fit a simple model for each source

source	measured as	Simulink block
encoder	step q	*Quantizer*, interval q
friction	start voltage V_c	*Dead Zone* $\pm V_c$ or *Coulomb & Viscous Friction*
other noise	spectrum of e	*Band-Limited White Noise* + low-pass filter (cut-off and size from the spectrum)

4. Add them to Model_pp_obs.slx

- *Quantizer* on the angle, before the controller (as the real encoder).
- *Dead Zone* on the voltage, after the saturation (friction).
- The filtered noise in the existing load input d (replace the 0.5 V step).
- *Transport Delay* 3 ms before the motor model.
- Fix the noise seed, so every run gives the same noise.

5. Validate and use

Run the same test (hold still, the square and the circle path) in simulation and on the bench, and compare them with `validate_controllers.m`: voltage ripple at rest (peak-to-peak and RMS), joint and tip errors, spectrum of u. When they are close, the model can be used to check a new design before the bench: for example the chatter limit (0.2 V) can be checked on the simulated u instead of with the formula, and PID and pole placement can be compared with the same noise.